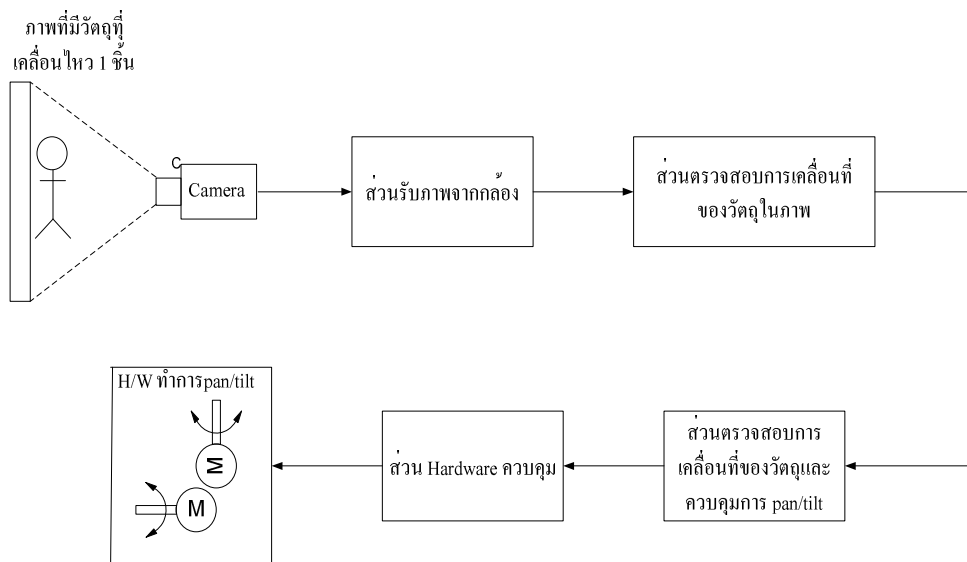


บทที่ 3

โครงสร้างและการออกแบบ

3.1 ภาพรวมของระบบ



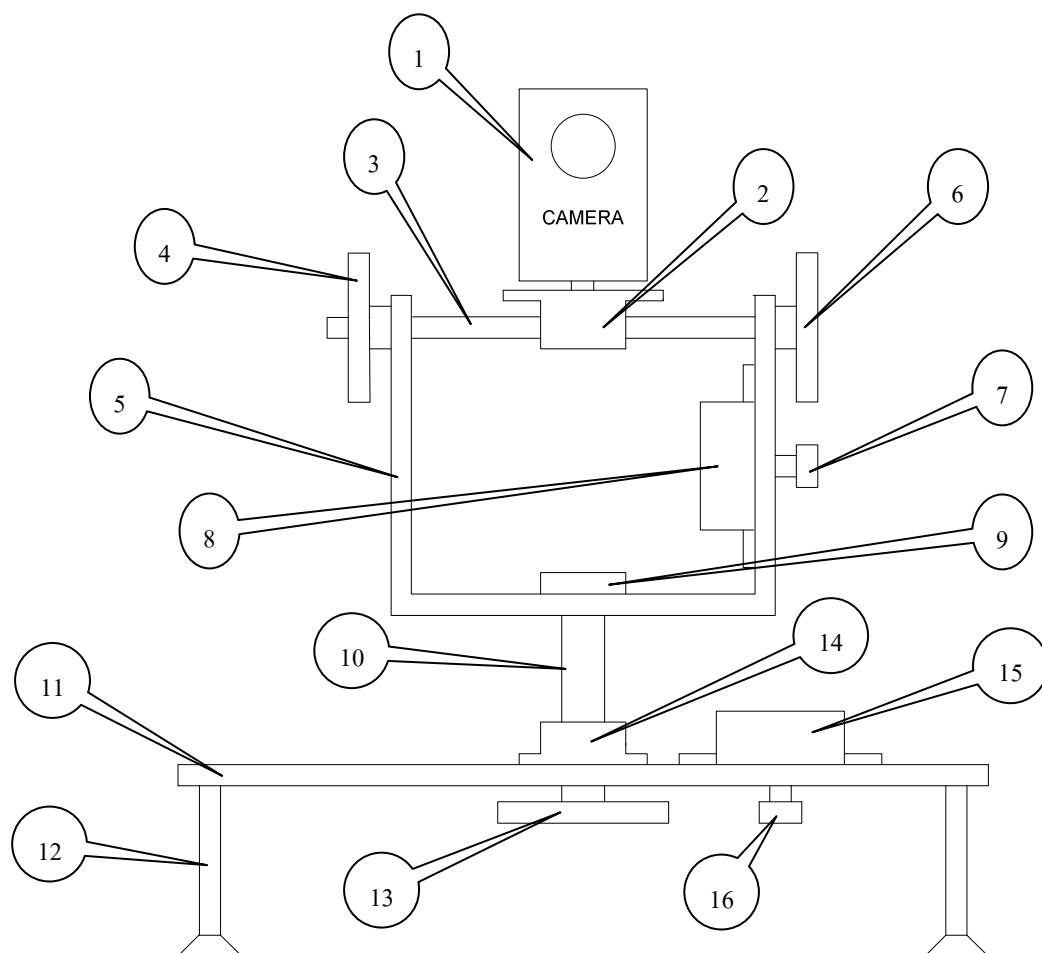
รูปที่ 3.1 แสดงภาพรวมของโครงการกล้องติดตามวัตถุ

จากรูปที่ 3.1 สามารถแบ่งภาพรวมของโครงการกล้องติดตามวัตถุเป็นส่วนย่อยๆ ได้ ดังนี้

1. ส่วนของกล้อง
2. ส่วนรับภาพจากกล้อง เป็นตัว Capture ภาพเข้ามา เพื่อนำภาพที่ได้ไปใช้ในส่วนถัดไป
3. ส่วนตรวจสอบการเคลื่อนที่ของวัตถุในภาพ จะนำภาพที่ได้จากการ Capture มาทำการตรวจสอบการเคลื่อนที่ของวัตถุ
4. ส่วนตรวจสอบเพื่อบังคับการ pan / tilt จะนำผลที่ได้จากการตรวจสอบการเคลื่อนที่ของวัตถุมาทำการตัดสินใจเพื่อที่จะบังคับให้ระบบควบคุมบังคับการเคลื่อนที่ของกล้อง ทำการ pan และ tilt ตามการเคลื่อนที่ของวัตถุ เพื่อให้วัตถุอยู่ในมุมมองของกล้องตลอดเวลา
5. ส่วน Hardware ควบคุม เป็นส่วนควบคุมมอเตอร์เพื่อทำให้กล้องเคลื่อนที่ไปตามผลจากส่วนที่ 4
6. ส่วน มอเตอร์ ทำการ pan / tilt ประกอบด้วยมอเตอร์ 2 ตัว

3.2 การออกแบบโครงสร้างของฐานกล้อง

ฐานกล้องสามารถเคลื่อนที่ได้ในลักษณะการ หันซ้าย-ขวา และ ก้ม-เงย ทั้งหมดได้ 4 ทิศทาง การทำงานจะถูกควบคุมโดยอุปกรณ์อิเล็กทรอนิกส์ และ โปรแกรมของไมโครคอนโทรลเลอร์ MCS-51 โครงสร้างทั้งหมดของฐานกล้องทำจากสแตนเลส และอะลูมิเนียม ซึ่งมีส่วนประกอบต่างๆ ดังนี้



รูปที่ 3.2 แสดงโครงสร้างของฐานกล้อง

ส่วนประกอบต่าง ๆ มีดังนี้

1. กล้องดิจิทัล
2. จุดยึดกล้องดิจิทัล
3. แกนหมุนปรับมุมก้มเงย
4. ตัวยึดแกนหมุน
5. โครงสร้างด้านบน
6. เฟืองแกนหมุนปรับมุมก้มเงย
7. เฟืองมอเตอร์
8. มอเตอร์ปรับมุมก้มเงย
9. ตัวยึดแกนหมุนกับโครงสร้างด้านบน
10. แกนหมุนซ้ายขวา
11. ฐานสแตนเลส
12. ขาตั้ง
13. เฟืองแกนหมุนซ้ายขวา
14. คลັบลูกปืน

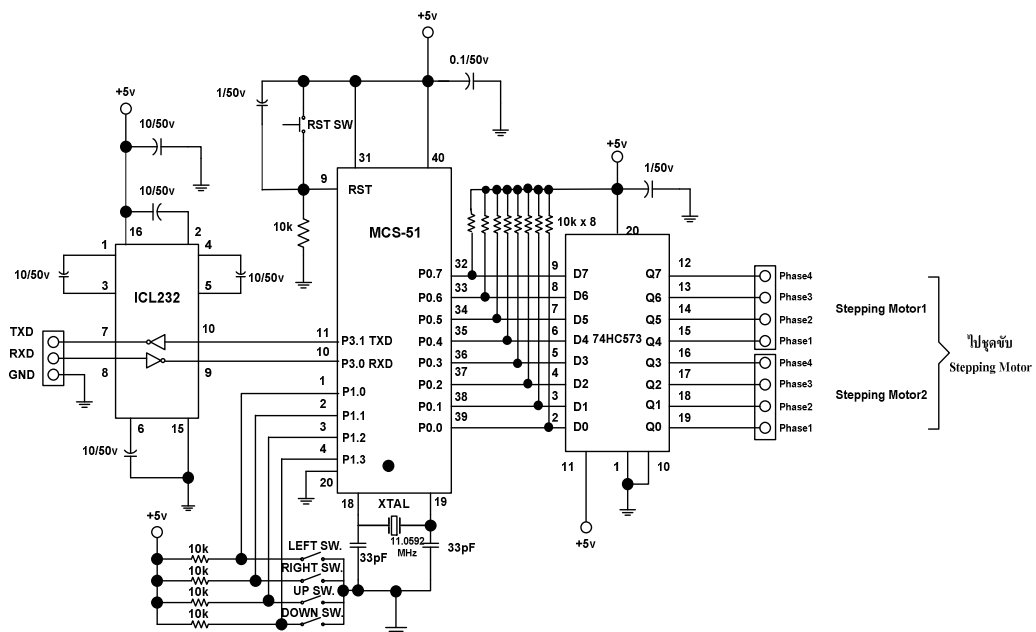
15. มอเตอร์ปรับหมุนซ้ายขวา

16. เฟืองมอเตอร์

เมื่อทำการประกอบชิ้นส่วนต่างๆ เสร็จแล้วก็จะมีลักษณะดังรูปที่ 3.2 ซึ่งแสดงตำแหน่งของอุปกรณ์ต่างๆ ที่ยึดไว้เรียบร้อยแล้ว อุปกรณ์ที่นำมาติดตั้งนั้น ได้แก่ กล้องดิจิทัล วงจรควบคุมการทำงาน MCS-51 และวงจรขับสเต็ปมอเตอร์ ในการออกแบบนั้นได้ออกแบบให้การติดตั้งอุปกรณ์ต่างๆ มีจุดยึดที่สามารถถอดออกได้สะดวก เพื่อประโยชน์ในการปรับปรุงประสิทธิภาพการทำงานต่อไป

3.3 การออกแบบวงจรควบคุมการทำงาน

การออกแบบวงจรควบคุมการทำงานนั้น ประกอบด้วยวงจรของไมโครคอนโทรลเลอร์ MCS-51 ที่มีโปรแกรมควบคุมสั่งงานสเต็ปมอเตอร์ให้หมุนก้มเงย และหมุนซ้ายขวา วงจรไมโครคอนโทรลเลอร์ MCS-51 นี้จะเชื่อมต่อกับวงจรบัฟเฟอร์และวงจรไครเวอร์ ซึ่งทำหน้าที่ขยายกระแสในการควบคุมการทำงานของมอเตอร์ให้มีประสิทธิภาพสูงสุด การควบคุมมอเตอร์นั้นสิ่งสำคัญคือกระแสที่จ่ายให้กับขดลวดของมอเตอร์ต้องคงที่เสมอ เพราะการหมุนแต่ละครั้งนั้นต้องมีความต่อเนื่องสม่ำเสมอ การออกแบบแหล่งจ่ายที่ไม่มีประสิทธิภาพจะทำให้ค่าตำแหน่งการหมุนผิดพลาดได้ การออกแบบวงจรจ่ายกระแสให้ระบบจึงจำเป็นต้องมีวงจรขยายกระแสที่มีประสิทธิภาพสูง ในการออกแบบโครงงานนี้ได้ออกแบบวงจรเพาเวอร์ซัพพลายโดยมีไอซีเรกูเลเตอร์เบอร์ 7805 ในการควบคุมแรงดันให้กับวงจรของไมโครคอนโทรลเลอร์ MCS-51 และมีไอซีเรกูเลเตอร์เบอร์ 7812 ในการควบคุมแรงดันให้กับวงจรไครเวอร์ ที่ทำหน้าที่ขยายกระแสในการควบคุมการทำงานของมอเตอร์

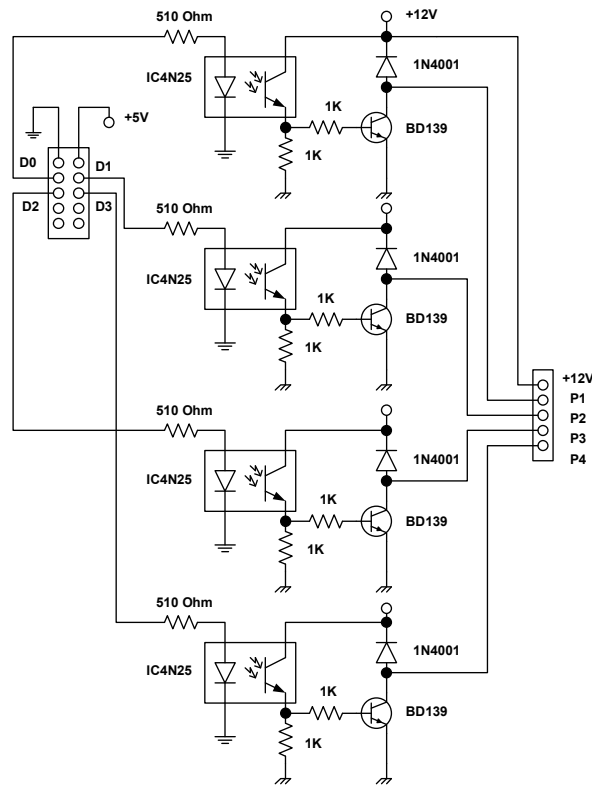


รูปที่ 3.3 แสดงส่วนไมโครคอนโทรลเลอร์ที่เชื่อมต่อกับวงจรบัฟเฟอร์และพอร์ตอนุกรมของคอมพิวเตอร์

จากรูปที่ 3.3 ตัวไมโครคอนโทรลเลอร์ MCS-51 จะเป็นตัวควบคุมและสั่งงานให้สเต็ปปีงมอเตอร์ทำการหมุนกัม-เงย และหมุนซ้าย-ขวา โดยจะรับข้อมูลการสั่งงานจากวงจรเชื่อมต่อพอร์ตอนุกรมของคอมพิวเตอร์ซึ่งมีไอซี ICL232 เป็นตัวทำหน้าที่ในการแปลงข้อมูลระดับ RS-232 ที่รับจากคอมพิวเตอร์ไปเป็นระดับทีทีแอล ก่อนที่จะถ่ายทอดไปยังไมโครคอนโทรลเลอร์ MCS-51 เพื่อทำการประมวลผลต่อไป (เนื่องจาก MCS-51 ไม่เข้าใจสัญญาณ RS-232) ได้ หน้าที่อีกอย่างหนึ่งของ ICL232 ก็คือแปลงข้อมูลส่งของไมโครคอนโทรลเลอร์ MCS-51 จากระดับทีทีแอลไปเป็นระดับของ RS-232 เพื่อใช้ตรวจสอบการทำงานของไมโครคอนโทรลเลอร์บนเครื่องคอมพิวเตอร์

พอร์ต 1 ของไมโครคอนโทรลเลอร์ใช้ในการรับอินพุตจากสวิทช์ 4 ตัวเพื่อจำกัดการหมุนกัม-เงย และหมุนซ้าย-ขวา จะเห็นได้ว่า P1.0 – P1.3 จะต่อกับสถานะลอจิก ‘1’ อยู่ตลอดเวลา เมื่อใดก็ตามที่มีอินพุตเข้ามาจากสวิทช์ตัวใดตัวหนึ่ง สถานะที่พอร์ต 1 ของบิตที่ต่อสวิทช์นั้นอยู่จะมีสถานะลอจิก ‘0’ นั่นแสดงว่าไม่สามารถที่จะสั่งงานให้สเต็ปปีงมอเตอร์หมุนต่อไปในทิศทางการหมุนเดิมได้อีก และจะใช้พอร์ต 0 ของไมโครคอนโทรลเลอร์ MCS-51 ในการส่งข้อมูลผ่านไอซีแลตช์ 74HC573 แล้วส่งต่อไปยังวงจร ไครเวอร์กระแสสูง ซึ่งแสดงดังรูปที่ 3.4 วงจรไครเวอร์ประกอบด้วยตัวออปโตทรานซิสเตอร์ IC4N25 จำนวน 4 ตัวตามเฟสของตัวสเต็ปปีงมอเตอร์ IC4N25 จะเริ่มทำงานเมื่อมีสัญญาณข้อมูล ‘1’ เข้ามาเป็นผลทำให้ทรานซิสเตอร์ BD139 ที่ต่ออยู่ทำงานขยายกระแสตามไปด้วย ทำให้มีกระแสไปขับสเต็ปปีงมอเตอร์ได้โดยวงจรไครเวอร์นี้สามารถขยายกระแสได้สูงสุด 2 แอมป์ ทั้งนี้ต้องเตรียมแหล่งจ่ายไฟฟ้าให้มีความสามารถในการจ่ายไฟฟ้าเพียงพอด้วย

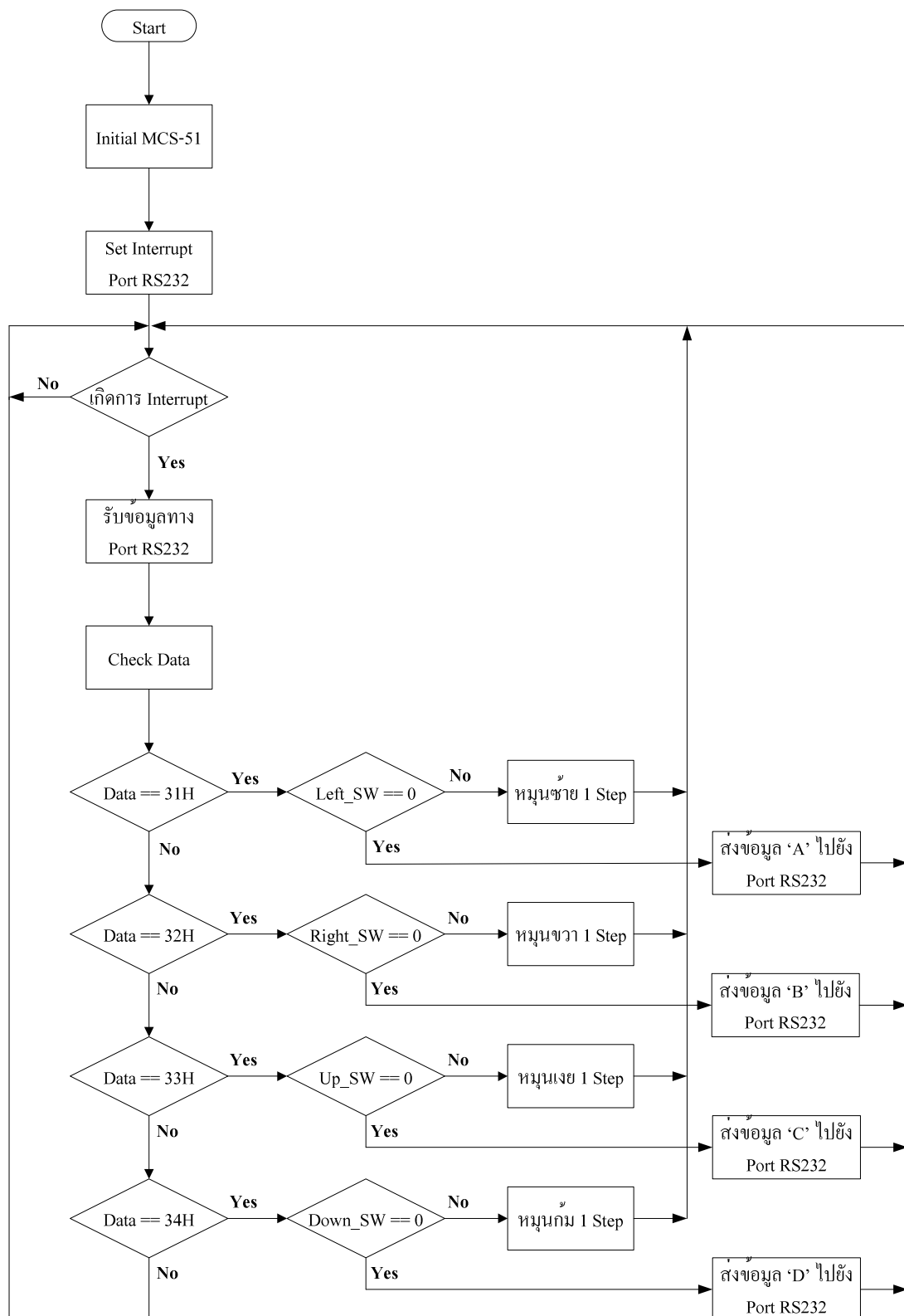
เมื่อต้องการให้สเต็ปปีงมอเตอร์ที่ทำหน้าที่หมุนซ้าย-ขวาทำการหมุน จะต้องส่งข้อมูล ‘1’ ไปเรียงไปตามลำดับจาก P0.0 ถึง P0.3 แล้ววนกลับมาที่ P0.0 ใหม่ หากต้องการให้สเต็ปปีงมอเตอร์กลับทิศทางก็ให้ส่งข้อมูลย้อนกลับ โดยเริ่มจาก P0.3 ก่อนแล้วสิ้นสุดรอบที่ P0.0 แล้ววนกลับมาที่ P0.3 ใหม่ ถ้าต้องการให้สเต็ปปีงมอเตอร์ที่ทำหน้าที่หมุนกัม-เงยทำการหมุน จะต้องส่งข้อมูล ‘1’ ไปเรียงไปตามลำดับทำนองเดียวกับสเต็ปปีงมอเตอร์ที่ทำหน้าที่หมุนซ้าย-ขวา แต่จะเรียงจาก P0.4 ถึง P0.7 แล้วกลับมาเริ่มใหม่ที่ P0.4 ใหม่อีก การให้สเต็ปปีงมอเตอร์หมุนกลับทิศทางทำได้โดยส่งข้อมูลเริ่มจาก P0.7 ถึง P0.4 แล้วกลับมาเริ่มใหม่ที่ P0.7 ใหม่ เมื่อวงจรไครเวอร์ได้รับข้อมูล “1” ผ่านมาทางไอซี 74HC573 วงจรก็จะทำงาน ทำให้เกิดกระแสไฟฟ้าไหลผ่านขดลวดของมอเตอร์ที่ต่ออยู่ ส่งผลให้เกิดการเคลื่อนที่ของแกนมอเตอร์ขึ้น



รูปที่ 3.4 แสดงวงจรไดรเวอร์สเต็ปมอเตอร์

3.4 การออกแบบโปรแกรมควบคุมสเต็ปมอเตอร์ของไมโครคอนโทรลเลอร์ MCS-51

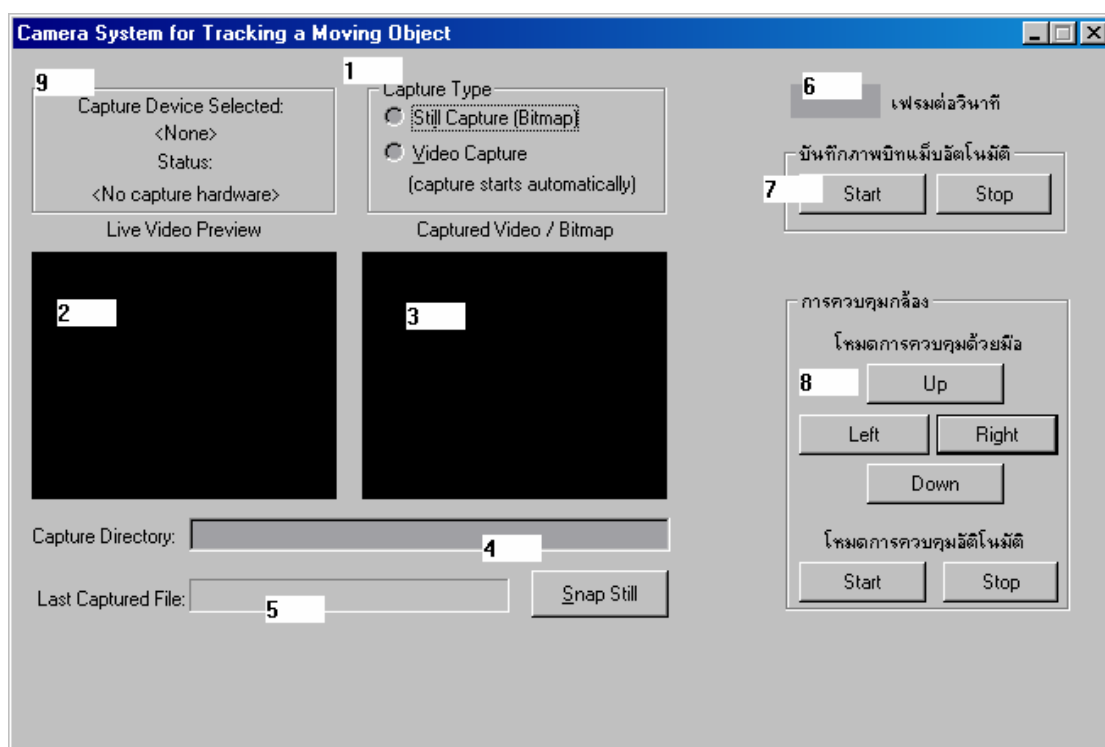
ในการเขียนโปรแกรมกระตุ้นและควบคุมการหมุนของมอเตอร์ให้เคลื่อนที่จะใช้แบบครึ่งสเต็ป (Half Step) เพราะสามารถเพิ่มจำนวนของสเต็ปต่อรอบได้อีกหนึ่งเท่าตัว และแรงบิดที่ได้จากการกระตุ้นแบบนี้จะเพิ่มมากขึ้นอีก เพราะช่วงสเต็ปมีระยะสั้นลง แต่ละสเต็ปเกิดแรงดึงจากขดลวด 2 ขดที่ถูกกระตุ้นพร้อมกัน ความถูกต้องของตำแหน่งก็มีเพิ่มมากขึ้น การเขียนโปรแกรมควบคุมสเต็ปมอเตอร์จะใช้หลักการอินเตอร์รัปต์ของพอร์ต RS232 เมื่อเกิดอินเตอร์รัปต์ขึ้นโปรแกรมจะนำข้อมูลที่รับมาทางพอร์ต RS232 มาตรวจสอบ ในที่นี้ ถ้าข้อมูลตัวเลข 1 (31H) เข้ามา 1 ตัว โปรแกรมจะสั่งงานให้หมุน สเต็ปมอเตอร์ไปทางซ้าย 1 สเต็ป ถ้าข้อมูลตัวเลข 2 (32H) เข้ามา 1 ตัว โปรแกรมจะสั่งงานให้หมุนสเต็ปมอเตอร์ไปทางขวา 1 สเต็ป ถ้าหากเป็นข้อมูลตัวเลข 3 (33H) เข้ามา 1 ตัว โปรแกรมจะสั่งงานให้หมุนสเต็ปมอเตอร์เวยขึ้น 1 สเต็ป และถ้าข้อมูลตัวเลข 4 (34H) เข้ามา 1 ตัว โปรแกรมจะสั่งงานให้หมุนสเต็ปมอเตอร์ก้มลง 1 สเต็ป สเต็ปมอเตอร์จะหมุนไปทางซ้ายได้เมื่อ Left_SW เท่ากับ 1 เมื่อใดก็ตามที่ Left_SW เท่ากับ 0 แสดงว่าสเต็ปมอเตอร์ไม่สามารถจะหมุนไปทางซ้ายได้อีก ในการหมุนไปทางขวา ก้ม และเวยก็จะ เป็นไปในลักษณะกัน เพียงแต่จะมี Right_SW เป็นตัวควบคุมการหมุนต่อไปได้ของการหมุนขวา Up_SW เป็นตัวควบคุมการหมุนต่อไปได้ของการเวยขึ้น และ Down_SW เป็นตัวควบคุมการหมุนต่อไปได้ของการห ก้มลง ถ้าหากว่าสเต็ปมอเตอร์ไม่สามารถหมุนต่อไปได้ไม่ว่าทางใด ตัวไมโครคอนโทรลเลอร์จะส่งข้อมูล กลับไปยังเครื่องคอมพิวเตอร์ทางพอร์ต RS232 เพื่อบอกว่าไม่สามารถหมุนมอเตอร์ต่อไปได้อีก Flowchart ของโปรแกรมแสดงดังรูปที่ 3.5



รูปที่ 3.5 แสดง Flowchart ของโปรแกรมควบคุมสปีดปั๊มมอเตอร์

3.5 การออกแบบหน้าจอ

หน้าจอโปรแกรมของระบบแสดงดังรูปที่ 3.6



รูปที่ 3.6 แสดงหน้าจอโปรแกรมของระบบ

รายละเอียดของหน้าจอมีดังนี้

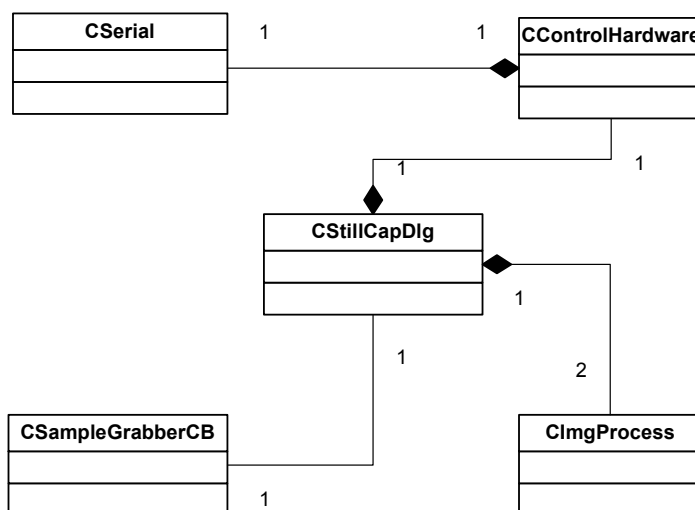
1. ประเภทของการแคปเจอร์ (Capture) ภาพจากกล้อง ซึ่งจะมียู่สองประเภทคือ การแคปเจอร์เป็นภาพนิ่งและการแคปเจอร์เป็นไฟล์วิดีโอ ถ้าเลือกเป็นโหมดของการแคปเจอร์เป็นไฟล์วิดีโอ จะเริ่มแคปเจอร์ทันที
2. ส่วนแสดงภาพเป็นสัญญาณภาพวิดีโอสดที่ได้ จากกล้อง
3. ส่วนแสดงภาพนิ่งที่ได้จากการถ่ายรูป หรือจากการบันทึกภาพบิตแมปอัตโนมัติ หรือจากโหมดการควบคุมกล้องอัตโนมัติ
4. ช่องสำหรับป้อนชื่อไดเรกทอรีที่ใช้เก็บไฟล์ที่ถูกแคปเจอร์
5. ส่วนแสดงชื่อไฟล์ที่ถูกแคปเจอร์ล่าสุด โดยชื่อไฟล์ที่ถูกบันทึกนั้นมีรูปแบบดังนี้
ปี-เดือน-วันที่-ชั่วโมง-นาที-วินาที-1/100 วินาที เช่น 2003-03-18-21-37-57-71, 2003-03-18-21-38-00-48 เป็นต้น
6. ช่องรับจำนวนเฟรมในการแคปเจอร์ในหนึ่งวินาที ซึ่งจะใช้ในการบันทึกภาพบิตแมปอัตโนมัติ และใช้ในโหมดการควบคุมกล้องอัตโนมัติ
7. ส่วนของการบันทึกภาพบิตแมปอัตโนมัติ ซึ่งเริ่มทำการแคปเจอร์โดยการกดปุ่มสแตท ถ้าจะหยุดก็ให้กดปุ่มสแตท

8. ส่วนของการควบคุมกล้อง ประกอบด้วยสองโหมด โหมดแรกคือโหมดการควบคุมด้วยมือ โหมดนี้ประกอบด้วยสี่ปุ่ม เพื่อใช้ในการควบคุมอุปกรณ์ คือปุ่มหมุนไปทางซ้าย ปุ่มหมุนไปทางขวา ปุ่มหมุนขึ้น และปุ่มหมุนลง สำหรับโหมดที่สองคือโหมดการควบคุมกล้องอัตโนมัติ ในโหมดนี้มีอยู่สองปุ่มด้วยกันปุ่มแรกเป็นปุ่มเริ่มการควบคุม และปุ่มที่สองเป็นปุ่มหยุดการควบคุม

9. ส่วนแสดงสถานะต่างๆ แบ่งเป็นสองส่วนย่อยดังนี้ ส่วนแรกจะแสดงว่าได้เลือกอุปกรณ์แคปเจอร์อะไร ส่วนที่สองแสดงสถานะของระบบว่ากำลังทำอะไรอยู่เช่น ทำการบันทึกภาพเมื่อบันทึกอัตโนมัติ หรือทำการควบคุมกล้องโหมดอัตโนมัติ หรือว่าทำการบันทึกเป็นไฟล์วิดีโออยู่

3.6 คลาสไดอะแกรม

คลาสไดอะแกรมของโปรแกรมแสดงดังรูป 3.7



รูปที่ 3.7 แสดงคลาสไดอะแกรมของโปรแกรม

3.7 ลักษณะของคลาสต่างๆ

3.7.1 คลาส CSerial ทำหน้าที่ติดต่อกับพอร์ตอนุกรม เมธอดที่สำคัญได้แก่ เมธอด Open ทำหน้าที่เปิดพอร์ตคอม 1 หรือ คอม 2 เมื่อต้องการส่งข้อมูลไปยังพอร์ตอนุกรมก็ต้องทำการเปิดพอร์ตก่อน เมื่อทำการเปิดพอร์ตสำเร็จจึงจะเริ่ม ทำการส่งข้อมูลโดย เมธอด SendData และทำการรับข้อมูลโดยใช้ เมธอด ReadData สำหรับแอตทริบิวต์และเมธอดของ คลาสนี้แสดงดังรูป 3.8

CSerial
#m_bOpened : bool
+CSerial() +~CSerial() +IsOpened() : bool +Open(in nPort : int, in bBaud : int) : bool +ReadData(out buffer : void, in limit : int) : int +SendData(in buffer : char, in size : int) : int

รูปที่ 3.8 แสดงลักษณะของ คลาส CSerial

3.7.2 คลาส CControlHardware ทำหน้าที่ ควบคุมอุปกรณ์บังคับการเคลื่อนที่ของกล้อง เมฆอดที่สามารถเรียกใช้ได้อีกคือ เมฆอด move โดยสามารถเรียกใช้ทั้งสองแบบด้วยกัน วิธีแรกมีไว้สำหรับ การควบคุม โหมดการบังคับด้วยมือ สามารถเรียกเมฆอดที่มีพารามิเตอร์สองตัว ส่วนวิธีที่สองมีไว้สำหรับการควบคุม โหมดการบังคับอัตโนมัติ โดยเรียกเมฆอดที่มีพารามิเตอร์ตัวเดียว พารามิเตอร์ตัวนี้เป็นเวกเตอร์บอกถึง ตำแหน่งจุดศูนย์กลางของวัตถุที่เคลื่อนที่อยู่ในภาพ โดยเมฆอด move ทั้งสองตัวนี้จะมีลักษณะการทำงานที่เป็นเทรค ลักษณะของคลาสนี้ดังรูปที่ 3.9

CControlHardware
-serial : CSerial -m_port_number : int
+CControlHardware(in port_number : int = 2) +move(in the_vector, in) : void +move(in msg : int, in step : int) : void -start() : void -start_controller() : void -turn_down() : bool -turn_up() : bool -turn_right() : bool -turn_left() : bool

รูปที่ 3.9 แสดงลักษณะของคลาสนี้ CControlHardware

3.7.3 คลาส CSampleGrabberCB ทำหน้าที่ติดต่อกับกล้อง อ่านภาพ แสดงภาพ และบันทึกเป็นไฟล์บิตแม็บ เมฆอดที่ใช้อ่านภาพจากกล้องคือ BufferCB โดยในการอ่านภาพจากกล้องนี้จะมีลักษณะการทำงานที่เป็นเทรค ถ้าบันทึกภาพลงในบัฟเฟอร์เสร็จแล้วก็จะส่งแมสเซจ ไปให้คลาส CSStillCapDlg เพื่อเอาภาพในบัฟเฟอร์ไปใช้งานต่อไป ส่วนการแสดงผลภาพจะใช้เมฆอด DisplayCapturedBits และการบันทึกภาพจะใช้เมฆอด CopyBitmap ลักษณะของคลาสนี้แสดงดังรูป 3.10

CSampleGrabberCB
+m_szCapDir +m_szSnappedName +IHeight : long +IWidth : long
+BufferCB(in dblSampleTime : double, in *pBuffer : unsigned char, in IBufferSize : long) +CopyBitmap(in dblSampleTime : double, in *pBuffer : unsigned char, in IBufferSize : long) +DisplayCaptureBits(in *pBuffer : unsigned char, in pbi)

รูปที่ 3.10 แสดงลักษณะของคลาสนี้ CSampleGrabberCB

3.7.4 คลาส CImgProcess ทำหน้าที่ประมวลผลภาพตามวิธีที่ได้ประยุกต์ขึ้นมา โดยจะทำการประมวลผลทีละสองภาพ ลักษณะการทำงานจะเป็นเทรค โดยจะเรียกผ่านเมธอด start ถ้าประมวลผลเสร็จแล้วจะส่ง แมสเชจจ์ไปให้ CStillCapDlg นำเอาผลไปใช้งานต่อไป ลักษณะของคลาส CImgProcess แสดงดังรูป 3.11

CImgProcess
#*st : signed char #*nd : unsigned char #m_nWidth : int #m_nHeight : int -m_nID : int -m_bRunning : bool -m_cRect +CImgProcess() +makeInfo(in *st : unsigned char, in *nd : unsigned char, in width : int, in height : int) +start() #getRight() #getLeft() #getBottom() #getTop() -IsSimilar(in x : int, in y : int)

รูปที่ 3.11 แสดงลักษณะของคลาส CImgProcess

3.7.5 คลาส CStillCapDlg ทำหน้าที่เป็นคลาสหลักของโปรแกรมของระบบได้แก่ คอยรับแมสเชจจ์ซึ่งระบุว่าได้ อ่านภาพจากกล้องแล้วเก็บลงในบัฟเฟอร์เรียบร้อยแล้ว จากคลาส CSampleGrabberCB เมื่อได้ภาพมาแล้วก็ทำการส่งให้คลาส CImgProcess ทำการประมวลผล จากนั้นก็คอยรับแมสเชจจ์แจ้งการประมวลผลภาพเสร็จ ที่เหลือก็เป็นส่วนของการนำผลลัพธ์จากการประมวลผลภาพมาวิเคราะห์ เพื่อจะสั่งให้อุปกรณ์ควบคุมกล้องหมุนตามวัตถุที่เคลื่อนที่ต่อไป หน้าที่สำคัญอีกอย่างหนึ่งคือทำหน้าที่ติดต่อกับหน้าจอ

CStillCapDlg
-*m_img1 : CImgProcess -*m_img2 : CImgProcess -*m_ctr_hw : CControlHardware *frame_st : unsigned char *frame_nd : unsigned char *frame_rd : unsigned char -initial_process_image() -automatical_controlling_process() -calculate_current_center_of_the_object() -calculate_size_of_the_object() -controlling_hardware() -get_direction() -image_process_from_the_frames() -mark_center_of_the_object() -on_proceed_bitmap() -start() #OnSaveStart() #OnSaveStop() #OnSnap() #OnTurnDown() #OnTurnLeft() #OnTurnRight() #OnTurnUp()

รูปที่ 3.12 แสดงลักษณะบางส่วน of คลาส CStillCapDlg